

Email Form Validation App — Requirements

Drafted 30 August 2026

Part 1 — General / Email Server Layer

1. The system is an email server that receives requests from clients by email.
2. Each incoming request carries a list of forms to be created or validated.
3. For validation requests, the client must supply the initial version of each form.
4. Each request must include a manifest defining the content requirements the forms are to be checked against.
5. The manifest is a free-text document. A simple parsing / normalisation step converts it into a set of discrete, individually checkable requirements before the AI editor sees it.
6. The email server performs light validation of every incoming email against the server's intake requirements.

7. If the email is valid, the client receives a confirmation reply stating that the request has been handed over to the AI editor.
 - *Recommended*: include the parsed requirement list in this reply, so the client can catch a misparsed requirement before comments are generated against it.
 8. If the email is invalid, the client receives a reply stating exactly what must be added or changed for the AI editor to be able to use it — e.g. missing document, missing manifest requirements.
 9. The email server hands off to a backend AI service that operates in two modes:
 - **Derivative** — validates supplied documents against the manifest requirements.
 - **Net-new** — composes a new form from mandatory inputs supplied by the client.
 10. Output in both modes is returned as Word documents carrying review comments.
 - *Derivative*: one comment per requirement per form, stating whether the requirement is met, a justification of the pass/fail result, and a suggestion of what to change in order to pass.
 - *Net-new*: comments showing how each predefined requirement was realised, with justification and a reference to the source requirement.
-

Part 2 – AI Editor

11. **Handover pattern.** Programmatic handover: each agent run is scoped to a specific set of requirements rather than the whole manifest.
12. **State.** The artifact under work lives in state external to any single agent run – a Python object the agent can modify at any point, including between runs.
13. **Reasoning tools** available to the agent, including:
 - image processing and image understanding
 - image cropping
14. **Editing tools** operate surgically on specific lines of the document. Edits are line-targeted; there is no full regeneration of the artifact.
15. **Output tools** apply the edits to the artifact and attach the justifications and source references described in requirement 10.
16. **Validation.** A Pydantic validator runs after each agent run and checks, for the requirement set that run owns:
 - every predefined requirement has an answer
 - every answer has a justification
 - every reference resolves to a real, existing source
17. **Retry.** On validator failure the error is passed back to the agent and the run is retried. Retries are capped at three attempts; after the third failure the requirement is marked as unverified in the returned comments rather than retried further.

Part 3 — Deferred / Open Concerns

D1. Input formats. v1 assumes Word documents only, on the basis that the target businesses work in Word. Scanned or PDF input would need an OCR pre-step. Deferred — note that line-targeted editing (req. 14) depends on a document model with addressable lines, so any new input format has to be normalised to that first.

D2. Job status and failure visibility. Once the client has received the "handed to the AI editor" confirmation, nothing currently defines what they see if the AI service crashes mid-run or the job runs long. Intended solution: the email layer should be able to query the AI service for job state and report back. Deferred.

D3. Requirement conflicts. Because each agent run sees only its own slice of the manifest, contradictory requirements in the manifest would not be detected by any run. A document could pass every individual check and still be internally incoherent. No owner assigned. Open.